

# Dissecting Refractor

## Asset storage in Battlefield 1942 and transformation for web rendering

DirectX 8, LZ01X, glTF 2.0, Three.js

A technical manual for developers. This document explains the binary asset formats used by the Refractor 2 engine in Battlefield 1942, identifies the parsing rules required to read them, and describes the pipeline that converts those assets into binary glTF files for rendering in web browsers.

## System architecture and asset structure

Battlefield 1942 was released in September 2002 using the Refractor 2 engine. The engine relied on DirectX 8 fixed-function graphics pipelines, flat compressed archives, imperative console scripts, binary geometry files with fixed vertex strides, and hierarchical skeletal animations.

Standard 3D software packages cannot open these files directly. Mesh files contain vertex buffers but no texture references. Weapon and soldier skeletons use mirrored coordinate frames along the Z axis. Animation clips store quaternion tracks with row-vector conventions and a 180-degree yaw offset. Terrain tiles contain baked sunlight, so standard WebGL scene lighting produces dark surfaces.

This guide explains the file formats used by the engine, outlines the failure modes encountered when parsing them, and details the extraction pipeline built in `tools/bf1942-models/` to convert these assets into binary glTF for Three.js.

### Analysis rule: Measure before decompiling

When an extracted asset renders incorrectly, inspect multiple files across the dataset before reading engine binaries in Ghidra. Write a Python script to scan the 33,000 meshes across vanilla and mod archives. Most discrepancies stem from coordinate system mismatches or mod-specific authoring variations.

### Pipeline overview from archive to WebGL

```
1. ARCHIVES (.rfa)      Unpack data payloads using LZ01X segment decompression.
2. OBJECT SCRIPTS (.con) Execute command streams to construct the parent-child scene graph.
3. GEOMETRY (.sm)       Read 16-bit indices, bounding boxes, collision hulls, and vertex strides.
4. SHADERS (.rs)        Match mesh material names to texture stages and alpha test flags.
5. TEXTURES (.dds/.tga) Decode DXT1, DXT3, DXT5, and BGRA rasters; bleed edges to prevent halos.
6. SKELETONS (.ske/.skn) Conjugate Z-mirrored bone matrices; align detached hand meshes to forearms.
7. ANIMATION (.baf)     Decode 7-channel fixed-point RLE tracks; conjugate quaternions; yaw root 180°.
8. EXPORT (glTF / .glb) Convert Refractor left-handed coordinates to WebGL right-handed coordinates.
9. BROWSER (Three.js)   Render with unlit materials, sRGB modulate-2x detail shaders, and Web Audio.
```

### The asset resolution contract

In Battlefield 1942, an asset is not a single file. An object definition relies on interdependent files spread across distinct archive directories:

```
+-----+ +-----+ +-----+
| Objects.con | --> | Geometries.con | --> | StandardMesh/*.sm |
| (Tree & Hierarchy) | | (Template Indirection) | | (Indexed Triangles) |
+-----+ +-----+ +-----+
                        |
                        v
+-----+ +-----+ +-----+
| Textures (*.dds) | <-- | RenderShader (*.rs) | <-----+
| (DXT1/3/5 Pixels) | | (Material Shaders) |
+-----+ +-----+ +-----+
```

1. `Objects.con` defines the object hierarchy, such as a tank hull parenting a turret, which parents a gun barrel.
2. `Geometries.con` resolves geometry aliases, mapping template names to geometry files.
3. `*.sm` stores vertex coordinates, surface normals, texture coordinates, and material slot names.
4. `*.rs` (RenderShader) matches material slot names to texture filenames.
5. `*.dds` and `*.tga` store the compressed or raw image rasters.

## Part 1 • Chapter 1

## Archives and scripts

All game assets reside within Refractor flat archives (`.rfa`). The format combines a header, a data payload, and an index table stored at the end of the file.

| Offset or field          | Type                  | Description                                                                                   |
|--------------------------|-----------------------|-----------------------------------------------------------------------------------------------|
| <code>0x00</code> Header | <code>char[28]</code> | Magic string: <code>Refractor2 FlatArchive 1.1</code> . When absent, base offset is zero.     |
| <code>data_size</code>   | <code>uint32</code>   | Length of the payload block in bytes.                                                         |
| <code>compressed</code>  | <code>uint32</code>   | Payload compression flag (1 for compressed, 0 for raw).                                       |
| <code>index_table</code> | Appended at EOF       | Located at <code>base_offset + data_size</code> . Contains entry count and directory records. |

### Entry tables and LZO1X segment decompression

The index table at `base_offset + data_size` begins with a 32-bit unsigned integer recording the entry count. Variable-length entry records follow. Each record contains the path length, the path string, compressed size, uncompressed size, and data offset:

```
def read_entries(data: bytes, base_offset: int, data_size: int):
    pos = base_offset + data_size
    entry_count, = struct.unpack_from("<I", data, pos)
    pos += 4
    entries = {}
    for _ in range(entry_count):
        name_len, = struct.unpack_from("<I", data, pos)
        name = data[pos+4 : pos+4+name_len].decode("latin-1").replace("\\", "/")
        comp_size, uncomp_size, offset = struct.unpack_from("<III", data, pos+4+name_len)
        pos += 4 + name_len + 12 + 12 # 12 bytes metadata plus 12 reserved bytes
        entries[name.lower()] = (offset, comp_size, uncomp_size)
    return entries
```

Compressed entries use the LZO1X algorithm. A compressed entry does not decompress in a single call. It begins with a 32-bit segment count, followed by descriptors for each segment:

```
import ctypes
_lzo = ctypes.CDLL("liblzo2.so.2")

def decompress_entry(raw_entry_data: bytes, uncompressed_size: int) -> bytes:
    segments_count, = struct.unpack_from("<I", raw_entry_data, 0)
    pos = 4
    out_buf = ctypes.create_string_buffer(uncompressed_size)
    out_offset = 0
    for _ in range(segments_count):
        seg_csize, seg_usize, seg_offset = struct.unpack_from("<III", raw_entry_data, pos)
        pos += 12
        chunk = raw_entry_data[seg_offset : seg_offset + seg_csize]
        written = ctypes.c_ulong(seg_usize)
        _lzo.lzo1x_decompress(chunk, seg_csize,
                               ctypes.byref(out_buf, out_offset), ctypes.byref(written), None)
        out_offset += seg_usize
    return bytes(out_buf)
```

### Mod search paths and case sensitivity

Refractor mods declare parent search paths in their `init.con` scripts using `game.addModPath`:

```
game.addModPath Mods/FHSW/
game.addModPath Mods/FH/
game.addModPath Mods/Bf1942/
```

The asset resolver checks the nearest child first and falls back to parents. Level archives also require underlaying. A mod level archive often supplies only vehicle spawns, relying on the base game level archive for terrain heightmaps, textures, and capture flags.

#### File system difference: Linux case sensitivity

Battlefield 1942 was developed on Windows, where paths are case-insensitive. Total conversion mods use mixed casing, such as `GCMOD/Archives`, `EoD/archives`, and `interstate/archives`. A direct path lookup on Linux fails when cases differ. The extractor scans directory entries with lowercase comparisons:

```
def find_archives(p): return next((c for c in p.iterdir() if c.name.lower() == "archives"), None)
```

## Stateful console scripts and child scoping

Game scripts use the `.con` extension. The engine processes these files as sequential command streams. It tracks an active template context as commands run:

```
ObjectTemplate.create Bundle ShermanComplex
ObjectTemplate.geometry Sherman_Hull_M1
ObjectTemplate.addTemplate ShermanTower
ObjectTemplate.setPosition 0/-0.8/0
```

### Scoping rule: Child instance transforms

In Refractor syntax, `setPosition` and `setRotation` calls placed after an `addTemplate` invocation apply to the child template just added, not to the parent bundle. Before any `addTemplate` line, position and rotation configure the parent. If a parser assigns position to the enclosing parent template, every child part collapses to the origin, causing turrets and wheels to render at the center of the vehicle hull.

## Vehicle inputs and accumulator spans

Movable vehicle parts connect to player inputs using `setInputToYaw`, `setInputToPitch`, and `setInputToRoll`. The script declares input indices or symbolic names. The declared angular span indicates whether a part spins continuously or holds a pose:

- Large spans (such as 2250 to 15000 degrees) identify continuous accumulators, including aircraft propellers, helicopter rotors, and wheels.
- Small spans (such as 2.0 degrees) identify deflection bounds, including chassis roll during acceleration and steering.

### Animation rule: Distinguishing continuous rotation from chassis lean

If an extractor treats every template of type `Engine` as a continuous spin, the chassis lean angle is integrated continuously over time. Because suspension components and wheels are children of the engine template, the running gear rotates around the tank hull. Setting an accumulator threshold at 360.0 degrees prevents this defect: values below 360.0 degrees are clamped poses rather than continuous rotation rates.

## Part 1 • Chapter 2

## Meshes, materials, and textures

Rigid geometry is stored in StandardMesh files (`.sm`). The format contains a fixed header, collision hulls, and an array of Level of Detail (LOD) representations.

| Byte offset  | Type     | Field        | Description                                                   |
|--------------|----------|--------------|---------------------------------------------------------------|
| 0x00         | uint32   | version      | Format version: 9 or 10.                                      |
| 0x04         | uint32   | reserved     | Zero.                                                         |
| 0x08 to 0x1F | float[6] | boundsMinMax | Axis-aligned bounding box coordinates.                        |
| 0x20         | uint8    | qflag        | Present only when <code>version &gt; 9</code> .               |
| Variable     | uint32   | colCount     | Count of collision hulls embedded in the file.                |
| Variable     | uint32   | lodCount     | LOD chain depth (1 for handheld items, up to 6 for vehicles). |

### Material descriptors and vertex strides

Within each LOD block, material descriptors are grouped together before the binary vertex and index buffers:

```
[LOD Header] -> [Mat 0 Desc] -> [Mat 1 Desc] -> [Mat 0 Verts] -> [Mat 0 Indices] -> [Mat 1 Verts] -> [Mat 1 Indices]
```

All material descriptors appear first. Vertex and index payloads follow in the same sequence.

```
# Material descriptor layout
name_len, = struct.unpack_from("<I", data, pos)
name = data[pos+4 : pos+4+name_len].decode("latin-1")
pos += 4 + name_len
u0, u1, u2, prim, flags, stride, vcount, icount, u3 = struct.unpack_from("<9I", data, pos)
pos += 36
```

Refractor uses two vertex layouts:

- **Stride 32 bytes** (Standard): Position (3 floats), Normal (3 floats), and Texture UV (2 floats).
- **Stride 40 bytes** (Static lightmapped): Position (3 floats), Normal (3 floats), Texture UV0 (2 floats), and Lightmap UV1 (2 floats).

### Index buffers and primitive types

DirectX 8 index buffers in Refractor use unsigned 16-bit integers. Meshes support two primitive types:

- `primitive == 4`: Triangle List. Every three indices form one triangle.
- `primitive == 5`: Triangle Strip. Each additional index after the first two creates a triangle, alternating vertex order:

```
for i in range(len(idx) - 2):
    a, b, c = idx[i], idx[i+1], idx[i+2]
    if a != b and b != c and a != c:
        tris.append((a, c, b) if i % 2 else (a, b, c))
```

#### Format rule: Unsigned index buffers

DirectX 8 index buffers use unsigned 16-bit integers (format `<H`). If unpacked as signed 16-bit integers (format `<h`), any mesh with more than 32,767 vertices produces negative index values. Large objects, such as battleships and industrial buildings, then render with stretched polygons connecting across distant coordinates.

### RenderShader definitions and texture lookup

Material names declared in the `.sm` file are matched to shader definitions in `StandardMesh/<mesh>.rs` or `Objects/.../Art/<mesh>.rs`:

```
subshader "Sherman_Hull_M1_Material0" "StandardMesh/Default" {
    texture "texture/sherma_I";
    alphaTest 0.5;
    twosided true;
}
```

Texture paths in shader scripts omit file extensions. The engine searches for a `.dds` file first, then falls back to `.tga`.

## DDS block compression and alpha channels

Textures use DirectDraw Surface compression formats:

- **DXT1:** Blocks of 4x4 pixels with 1-bit alpha or opaque color punchthrough.
- **DXT3:** Explicit 4-bit alpha channels, used for sharp transparency edges.
- **DXT5:** Interpolated 8-step alpha ramps, used for gradients and smoke.

## Part 1 • Chapter 3

## Skeletons, skins, and animations

Vehicles use rigid node trees positioned through script offsets. Human soldiers and animated weapons use a three-part deformation system:

```
[1. .SKE (Armature)] =====> [2. .SKN (Skin Weights)] <===== [3. .SM (3D Geometry)]
      |                               |
      v (Drives Joints)              v (Deforms Vertices)
[4. .BAF (Animation Tracks)] =====> [FINAL POSED MODEL]
```

## Rest-pose skeletons (.ske)

A `.ske` file defines the rest-pose bone hierarchy:

```
uint32 version (1)
uint32 boneCount
per bone:
  uint16 nameLen
  char name[nameLen]
  int16 parentIndex (-1 for root)
  float matrix[12] (Row-major 3x4: [R00 R01 R02 Tx, R10 R11 R12 Ty, R20 R21 R22 Tz])
```

## Geometry rule: Skeleton coordinate reflection

A `.ske` file stores bones reflected across the Z axis relative to the `.sm` geometry it controls. In `UsSoldier.ske`, the head bone sits at  $z = -1.51$  m, whereas the head mesh geometry sits at  $z = +1.56$  to  $1.87$  m. Reading the skeleton without reflection places weapon magazines and bolt handles behind the weapon body.

## Skin bindings (.skn)

A `.skn` file assigns vertex weights to bones:

```
float restPos[3]
uint8 influenceCount
per influence:
  uint16 boneIndex, float weight, float boneLocalOffset[3]
```

The bind pose satisfies:  $\text{rest} = R_{\text{bone}} * \text{offset} + T_{\text{bone}}$ .

## Keyframed bone animation clips (.baf)

Soldier animations blend a lower-body clip (such as `Lb_Stand`) with an upper-body clip (such as `3PStandAimUpperThompson.baf`).

| Header field | Type   | Description                                 |
|--------------|--------|---------------------------------------------|
| version      | uint32 | Always 3 in vanilla archives.               |
| boneCount    | uint16 | Count of animated bones.                    |
| frameCount   | uint32 | Frame count of the animation.               |
| precision    | uint8  | Fractional bits for position channels only. |

Each bone stores seven channels: Quaternion (x, y, z, w) followed by Position (x, y, z). Channels are stored as 16-bit signed fixed-point integers compressed with run-length encoding:

- `control & 0x80`: Hold run. Read one value and repeat it for `control & 0x7F` frames.
- Else: Literal run. Read `control & 0x7F` distinct values.

## Decompression rule: Position and quaternion precision

The header byte `precision` indicates the fractional bit count for position values:  $\text{pos} = \text{val} / (2^{\text{precision}})$ . This scaling factor does not apply to quaternions. Quaternions consistently use 1.15 fixed-point format:  $\text{quat} = \text{val} / 32768.0$ . Applying the position scaling factor to quaternions corrupts rotations when precision drops below 15.

## The animation state machine (AnimationStates.con)

Weapon templates do not reference animation clips directly. The engine resolves clips by replaying `animations/AnimationStates.con`:

- `copyState2` commands replace generic weapon placeholders with weapon names.

- `copyState` commands assign fallback clips: the K98 uses the British Lee-Enfield No. 4 aim clip; the Panzerschreck uses the Bazooka clip; the Walther P38 uses the Colt 1911 clip.
- Weapon skeletons root at a bone named `Bip01 R Hand`, attaching directly to the soldier skeleton hand bone.

## Part 1 • Chapter 4

## Terrains, lighting, and audio

### Elevation grids and pre-lit terrain tiles

Terrains in Battlefield 1942 are defined by two data sources:

- `Heightmap.raw`: 16-bit unsigned elevation samples on a 4-meter grid:  $\text{elevation} = (\text{sample} / 65535) * 256 * \text{yScale}$ .
- `Tx*.dds`: 256-meter square terrain color tiles.

#### Lighting rule: Pre-calculated terrain illumination

The level editor pre-calculated sunlight, ambient color, and building cast shadows directly into the pixels of the terrain tiles. The original game rendered terrain without dynamic directional lights. Adding standard WebGL directional and hemisphere lights produces duplicate lighting, causing terrain to appear over-darkened and discolored.

### Static mesh lightmaps

Structures, bridges, and bunkers use Stride 40 vertices containing a secondary UV channel (`uv1`) mapped to pre-calculated lightmap images (such as `radarbun_m1_406-75-761.png`). These textures store direct sunlight and shadow values, which the engine multiplies by the diffuse texture at runtime.

### Sound scripts and polyline area sounds

Audio files are stored in `Sound.rfa` and level archives as uncompressed 16-bit PCM WAV files. Playback rules are defined in Sound Script Configuration (`.ssc`) files:

```
newPatch
load @ROOT/Sound/@RT/Water_waves.wav
loop
minDistance 10.0
maxDistance 150.0
volume 0.8
priority 5
```

Levels define ambient loops in `Sounds/Environment.con` and shoreline audio as polylines tracing the water boundary. Sound emanates from the point on the polyline closest to the camera.

## Part 2 • Chapter 5

## glTF generation and coordinate conversion

To render Refractor assets in web browsers, the pipeline in `tools/bf1942-models/` extracts geometry, transforms coordinate spaces, and compiles assets into binary glTF 2.0 (`.glb`) files.

### Extraction pipeline modules

The extraction tools are structured as focused modules:

- `rfa.py`: Reads archives, caches index tables, and performs LZO1X decompression.
- `con.py`: Parses script syntax and tracks template trees.
- `stdmesh.py`: Reads vertex records, material descriptors, and index buffers.
- `rs.py`: Parses shader scripts to map material slots to texture names.
- `gltf.py`: Constructs binary glTF containers with vertex buffers and accessors.
- `assemble.py`: Traverses template hierarchies and builds complete vehicle models.
- `pose.py`: Assembles soldier skeletons, skins, and weapon attachments.
- `level.py` and `terrain.py`: Extract terrain meshes and place map objects.

### Coordinate system handedness conversion

Refractor uses a left-handed coordinate system (+X right, +Y up, +Z forward). glTF and WebGL use a right-handed coordinate system (+X right, +Y up, -Z forward).

Applying a negative scale on the root node inverts polygon winding, corrupts surface normal vectors, and breaks back-face culling. The pipeline bakes coordinate transformation into the vertex buffers:

```
# Coordinate conversion in gltf.py
positions = [(x, y, -z) for (x, y, z) in sm.positions]
normals    = [(nx, ny, -nz) for (nx, ny, nz) in sm.normals]
indices    = [idx for i in range(0, len(raw_indices), 3)
              for idx in (raw_indices[i], raw_indices[i+2], raw_indices[i+1])] # Reverse winding
```

For vehicle Euler rotations (Yaw, Pitch, Roll):  $R = R_y(-yaw) * R_x(-pitch) * R_z(roll)$ . Inverting the Z axis reverses the direction of rotations around X and Y while preserving Z rotation.

### Assembling binary glTF containers

When exporting to glTF 2.0:

- **Sub-mesh primitives:** An `.sm` part with multiple materials is mapped to a single glTF node containing multiple primitives. Each primitive references its own material definition and index slice.
- **Collision geometry isolation:** Collision hulls from the `.sm` file are omitted from the visible render tree and stored in `mesh.extras.collision` to prevent collision shapes from drawing over the vehicle body.

### Texture export and alpha dilation

Textures are decoded from DXT rasters and exported as PNG or WebP images.

Cutout textures, such as foliage, wire fences, and tank tracks, frequently store zero values (black) in transparent pixels. When Three.js applies bilinear interpolation at the boundary between opaque and transparent pixels, the black color bleeds into the edge, creating a dark outline.

The exporter runs an alpha dilation pass: for any pixel with an alpha value of 16 or lower, it copies the RGB values from the nearest opaque neighbor. This eliminates dark halos around cutout boundaries.

## Part 2 • Chapter 6

## Rigging and animation in the browser

### Skeleton rest matrix Z-mirror conjugation

Because skeletons in `.ske` files are mirrored along the Z axis relative to mesh geometry, the exporter conjugates rotation matrices and translations using reflection matrix `S = diag(1, 1, -1)`:

```
def conjugate_ske(rot, trans):
    rot_c = (( rot[0][0],  rot[0][1], -rot[0][2]),
              ( rot[1][0],  rot[1][1], -rot[1][2]),
              (-rot[2][0], -rot[2][1],  rot[2][2]))
    trans_c = (trans[0], trans[1], -trans[2])
    return rot_c, trans_c
```

This transformation aligns skeleton joint positions with the mesh coordinate frame.

### Quaternion conjugation and root bone yaw

Animation clips require two adjustments before playback in WebGL:

1. **Quaternion conjugation:** Refractor stores row-vector quaternions. To convert to standard column-vector matrices, the exporter conjugates each quaternion:  $(x, y, z, w) \rightarrow (-x, -y, -z, w)$ .
2. **Root joint yaw:** The clip coordinate space is rotated 180 degrees around Y relative to mesh world space. The root bone transform is pre-multiplied by `Ry(180) = diag(-1, 1, -1)`.

### Aligning detached hand meshes

Soldier models use separate meshes for the body, head, and hands. The body and head share an origin, but hand meshes were exported in separate coordinate frames.

Both the body and hand meshes share the forearm bone. The pipeline finds vertices with a weight of 1.0 on the forearm bone in both meshes, calculates the rigid coordinate offset between the two frames, and transforms the hand vertices to connect with the sleeve.

### Stance crossfading and action bundling

Rather than writing separate files for each posture, `pose.py` packages standing, crouching, and prone stances into a single `.glb` file as single-frame animation actions:

- `stand`: Upper-body aim clip combined with `Lb_Stand`.
- `crouch`: Upper-body aim clip combined with `Lb_Crouch`.
- `lie`: Upper-body aim clip combined with `Lb_Lie`.

The browser uses Three.js `AnimationMixer` to crossfade between stances over 0.25 seconds, producing continuous posture transitions without joint discontinuities.

## Part 2 • Chapter 7

## Three.js browser reconstruction

### Unlit terrain shading

Because terrain tiles already contain baked illumination and shadows, terrain meshes use `MeshBasicMaterial` in Three.js without scene light calculations. This preserves the original color balance without double lighting.

### The sRGB modulate-2x detail shader

Refractor blends a repeating 512-pixel detail texture across each terrain tile using the formula: `FinalColor = clamp(2.0 * TileColor * DetailColor, 0.0, 1.0)`. Neutral gray (128, 128, 128) leaves the surface unchanged.

#### Color space rule: Modulate-2x detail calculation

Modern WebGL applies linear color space conversions to textures. In linear space, neutral gray 0.5 becomes approximately 0.218. Multiplying by 2.0 yields 0.43 instead of 1.0, making the terrain 2.3 times darker than intended. The viewer sets `detailTex.colorSpace = THREE.NoColorSpace` and performs the calculation in sRGB space through an `onBeforeCompile` GLSL hook:

```
// Three.js onBeforeCompile fragment hook
diffuseColor.rgb = bfFromSrgb( min( vec3(1.0),
    2.0 * bfToSrgb( diffuseColor.rgb ) * texture2D( tDetail, vMapUv * detailRepeats ).rgb ) );
```

### Lightmap sampling and ambient floor injection

Static buildings sample lightmap atlases using their second UV channel.

#### Texture rule: Lightmap coordinate orientation

glTF images load with `flipY = false`, whereas `THREE.TextureLoader` defaults to `flipY = true`. Sampling an inverted lightmap projects roof shadows onto interior floors. Setting `lightmap.flipY = false` aligns the lightmap coordinates with the mesh geometry.

The material fragment shader applies the level's ambient light floor to keep shadowed surfaces visible:

```
// Patched fragment shader for static lightmaps
vec3 lmTexel = texture2D( lightMap, vLightMapUv ).rgb;
reflectedLight.indirectDiffuse += diffuseColor.rgb * max( lmTexel * 2.0, ambientFloor );
```

### Interactive vehicle controls and projectile ballistics

In the viewer, vehicle parts react to user controls:

- Rotational parts with spans above 360 degrees update continuously using their rotational speed.
- Parts with small spans clamp to their authored deflection limits.
- Recoil offsets move gun barrels backward along their local Z axis upon firing.
- Muzzle smoke particles drift backward along the barrel using authored drift velocities.
- Weapons spawn typed projectiles: kinetic bullets draw tracer lines; artillery shells and rockets render 3D meshes with gravity drop and acceleration.

## Part 2 • Chapter 8

## Spatial audio in the browser

Level audio is spatialized using the Web Audio API with `AudioContext` and `PannerNode` (HRTF model).

### Looping voice deduplication

Levels often define multiple shoreline polylines that reference the same audio file, such as `water_waves.wav`.

#### Audio rule: Looping voice deduplication

When multiple polylines each play an independent audio stream, small differences in decoding time cause streams to play out of phase. Playing identical audio streams 100 milliseconds apart through HRTF panners causes acoustic comb filtering and phase cancellation. The viewer resolves this by running one looping audio voice per unique WAV file, updating its coordinates to the point on any active polyline nearest to the camera.

### Custom distance attenuation curves

The Web Audio `PannerNode` applies an inverse distance model by default.

#### Audio rule: Distance attenuation curves

Combining default Web Audio distance attenuation with the linear volume curves from `.SSC` scripts attenuates sound twice, cutting off shoreline audio near the water boundary. Calling `panner.setRolloffFactor(0)` disables Web Audio automatic dropoff, allowing the game's authored `minDistance` and `maxDistance` thresholds to govern attenuation directly.

## Part 3 • Chapter 9

## Reference specifications

## File formats and magic bytes

| Extension | Domain    | Identifier or format       | Parsing rule                                                    |
|-----------|-----------|----------------------------|-----------------------------------------------------------------|
| .rfa      | Archive   | Refractor2 FlatArchive 1.1 | Index table at EOF; LZ01X segment decompression.                |
| .sm       | Geometry  | Version 9 or 10            | Descriptors precede buffers; unsigned 16-bit indices.           |
| .rs       | Shader    | ASCII script               | Maps material slot names to texture paths.                      |
| .ske      | Skeleton  | Version 1                  | 3x4 rest matrices; conjugate with $S = \text{diag}(1, 1, -1)$ . |
| .skn      | Skin      | Version 1                  | Vertex positions, bone indices, and local offsets.              |
| .baf      | Animation | Version 3                  | 7-channel RLE; conjugate quaternions; yaw root 180°.            |
| .con      | Script    | ASCII script               | setPosition applies to preceding child template.                |
| .ssc      | Audio     | ASCII script               | Distance thresholds, loop flags, and sample links.              |

## Coordinate transformation reference

| Coordinate space           | Handedness    | Axis orientation            | Conversion to WebGL glTF                                                                  |
|----------------------------|---------------|-----------------------------|-------------------------------------------------------------------------------------------|
| Refractor engine           | Left-handed   | +X Right, +Y Up, +Z Forward | Invert Z axis; reverse triangle winding order.                                            |
| glTF 2.0 / WebGL           | Right-handed  | +X Right, +Y Up, -Z Forward | Target coordinate system.                                                                 |
| Refractor skeleton (.ske)  | Mirrored left | +X Right, +Y Up, -Z Forward | Conjugate: $R' = S \cdot R \cdot S$ , $t' = S \cdot t$ with $S = \text{diag}(1, 1, -1)$ . |
| Refractor animation (.baf) | Yawed left    | +X Left, +Y Up, -Z Backward | Conjugate quaternion: $(-x, -y, -z, w)$ ; yaw root joint 180°.                            |

## Defect triage checklist

| Visual symptom                             | Root cause                                                       | Resolution                                                                     |
|--------------------------------------------|------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Child part missing or positioned at origin | setPosition applied to parent template                           | Assign transform to the preceding child instance.                              |
| Tank suspension rotates around hull        | Chassis lean treated as continuous angular rate                  | Apply 360-degree threshold to distinguish accumulators from poses.             |
| Stretched polygons on large meshes         | Index buffer unpacked as signed 16-bit integers                  | Unpack index buffer using unsigned short format (<H).                          |
| Weapon parts positioned behind weapon body | Skeleton rest matrix read without Z reflection                   | Conjugate rotation and translation matrices with $S = \text{diag}(1, 1, -1)$ . |
| Inverted soldier joint rotations           | Animation quaternion read without conjugation                    | Conjugate quaternion: $(-x, -y, -z, w)$ .                                      |
| Terrain renders too dark                   | Modulate-2x detail texture decoded in linear space               | Set NoColorSpace on detail map and compute in sRGB space.                      |
| Roof shadows appear on interior floors     | Lightmap texture sampled with inverted V axis                    | Set lightmap.flipY = false.                                                    |
| Rocket renders as faint line               | Rocket projectile missing geometry definition                    | Export projectile mesh and simulate flight acceleration.                       |
| Comb filtering on shoreline audio          | Duplicate out-of-phase audio loops playing on polylines          | Use one looping voice per unique WAV file across polylines.                    |
| Shoreline audio cuts off abruptly          | Web Audio inverse attenuation applied in addition to script ramp | Set panner.setRolloffFactor(0).                                                |